1A

```
# 1. Import libraries
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

# 2. Load training and testing data
train_data = pd.read_csv("F:/DL/LP IV Updated Datasets/FNN and CNN/HandWritten
MNIST/mnist_train.csv")
test_data = pd.read_csv("F:/DL/LP IV Updated Datasets/FNN and CNN/HandWritten
MNIST/mnist_test.csv")

X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# # 2. Load training and testing data
# (X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

# # Normalize pixel values
# X_train = X_train / 255.0
# X_test = X_test / 255.0

# # Flatten 28x28 images into 784 values
# X_train = X_train.reshape(-1, 784)
# X_test = X_test.reshape(-1, 784)

# 3. Define the FNN model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

# 4. Compile using SGD
model.compile(
    optimizer=tf.keras.optimizers.SGD(),
```

```
    loss = tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"]
)

# Train for 11 epochs
history = model.fit(
    X_train, y_train,
    epochs=11,
    batch_size=32,
    validation_split=0.1
)

# 5. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# 6. Plot loss and accuracy
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["loss"])
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.subplot(1, 2, 2)
plt.plot(history.history["accuracy"])
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.tight_layout()
plt.show()
```

1B

```
# 1. Import libraries
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

# 2. Load training and testing data
train_data = pd.read_csv("F:/DL/LP IV Updated Datasets/FNN and CNN/Fashion
MNIST/fashion-mnist_train.csv")
test_data = pd.read_csv("F:/DL/LP IV Updated Datasets/FNN and CNN/Fashion
MNIST/fashion-mnist_test.csv")

# Separate features and labels
X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# # 2. Load training and testing data
# (X_train, y_train), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

# # Normalize pixel values
# X_train = X_train / 255.0
# X_test = X_test / 255.0

# # Flatten 28x28 images into 784 values
# X_train = X_train.reshape(-1, 784)
# X_test = X_test.reshape(-1, 784)

# 3. Define the FNN model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

# 4. Compile the model using SGD
model.compile(
```

```
    optimizer=tf.keras.optimizers.SGD(),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"]
)

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=11,
    batch_size=32,
    validation_split=0.1
)

# 5. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# 6. Plot training loss and accuracy
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["loss"])
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.subplot(1, 2, 2)
plt.plot(history.history["accuracy"])
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.tight_layout()
plt.show()
```

1C

```
# 1. Import libraries
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

# 2. Load the dataset
data = pd.read_csv("F:/DL/LP IV Updated Datasets/FNN and CNN/CIFAR10/cifar-10.csv")

# Separate features and labels
X = data.drop("label", axis=1).values
y = data["label"].values

# Normalize pixel values
X = X.astype("float32") / 255.0

# Split into training and testing data
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# # 2. Load training and testing data
# (X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()

# # Convert labels into 1D arrays
# y_train = y_train.flatten()
# y_test = y_test.flatten()

# # Normalize pixel values
# X_train = X_train.astype("float32") / 255.0
# X_test = X_test.astype("float32") / 255.0

# # Flatten images into 3072 values
# X_train = X_train.reshape(-1, 3072)
# X_test = X_test.reshape(-1, 3072)

# 3. Define the FNN model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(3072,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
```

```
])

# 4. Compile the model
model.compile(
    optimizer=tf.keras.optimizers.SGD(),  # SGD optimizer
    # optimizer=tf.keras.optimizers.Adam(),  # Adam optimizer
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"]
)

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=11,
    batch_size=32,
    validation_split=0.1
)

# 5. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# 6. Plot training loss and accuracy
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["loss"])
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.subplot(1, 2, 2)
plt.plot(history.history["accuracy"])
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.tight_layout()
plt.show()
```

1E

```
# 1. Import libraries
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

# 2. Load training and testing data
train_data = pd.read_csv("F:/DL/LP IV Updated Datasets/FNN and CNN/Sign Language
MNIST/sign_mnist_train.csv")
test_data = pd.read_csv("F:/DL/LP IV Updated Datasets/FNN and CNN/Sign Language
MNIST/sign_mnist_test.csv")

# Separate features and labels
X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

# Normalize pixel values
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# 3. Define the FNN model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(25, activation="softmax")
])

# 4. Compile the model
model.compile(
    optimizer=tf.keras.optimizers.SGD(),  # SGD optimizer
    # optimizer=tf.keras.optimizers.Adam(),  # Adam optimizer
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"]
)

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=11,
    batch_size=32,
```

```
    validation_split=0.1
)

# 5. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# 6. Plot training loss and accuracy
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["loss"])
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.subplot(1, 2, 2)
plt.plot(history.history["accuracy"])
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.tight_layout()
plt.show()
```

2A

```
# 1. Import libraries
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# 2. Load and preprocess the data
data = pd.read_csv(
    "F:/DL/LP IV Updated Datasets/FNN and CNN/CIFAR10/cifar-10.csv"
)

# Separate features and labels
X = data.drop("label", axis=1).values
y = data["label"].values

# Normalize pixel values
X = X.astype("float32") / 255.0

# Reshape images into 32x32 RGB format
X = X.reshape(-1, 32, 32, 3)

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# # 2. Load and preprocess the data
# (X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()

# # Normalize pixel values
# X_train = X_train / 255.0
# X_test = X_test / 255.0

# 3. Define the CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3)),

    tf.keras.layers.Conv2D(32, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),
```

```
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
# Compile the model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
# 4. Train the model
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1
)

# 5. Estimate model performance
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# Plot training loss and accuracy
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["loss"], label="Training Loss")
# plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history["accuracy"], label="Training Accuracy")
# plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.tight_layout()
plt.show()
```

2B

```
# 1. Loading and preprocessing the image data
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

train_data = pd.read_csv(
    "F:/DL/LP IV Updated Datasets/FNN and CNN/HandWritten MNIST/mnist_train.csv"
)

test_data = pd.read_csv(
    "F:/DL/LP IV Updated Datasets/FNN and CNN/HandWritten MNIST/mnist_test.csv"
)

X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# (X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

# X_train = X_train / 255.0
# X_test = X_test / 255.0

# X_train = X_train.reshape(-1, 28, 28, 1)
# X_test = X_test.reshape(-1, 28, 28, 1)


# 2. Defining the model's architecture
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),

    tf.keras.layers.Conv2D(32, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
```

```
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)


# 3. Training the model
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)


# 4. Estimating the model's performance
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")

# Plotting accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Plotting loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

2C

```
# 1. Loading and preprocessing the image data
import tensorflow as tf
import pandas as pd
import matplotlib.pyplot as plt

train_data = pd.read_csv(
    "F:/DL/LP IV Updated Datasets/FNN and CNN/Sign Language MNIST/sign_mnist_train.csv"
)

test_data = pd.read_csv(
    "F:/DL/LP IV Updated Datasets/FNN and CNN/Sign Language MNIST/sign_mnist_test.csv"
)

X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)


# 2. Defining the model's architecture
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),

    tf.keras.layers.Conv2D(32, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(25, activation="softmax")
])

model.compile(
```

```
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)


# 3. Training the model
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)


# 4. Estimating the model's performance
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")


# Plotting accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()


# Plotting loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

3A

```
import pandas as pd
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report

# 1. Load dataset
data = pd.read_csv("F:/DL/LP IV Updated Datasets/AutoEncoder/CreditCard/creditcard.csv")

# 2. Preprocess data
X = data.drop("Class", axis=1)
y = data["Class"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale Time and Amount
scaler = StandardScaler()
X_train[["Time", "Amount"]] = scaler.fit_transform(
    X_train[["Time", "Amount"]]
)
X_test[["Time", "Amount"]] = scaler.transform(
    X_test[["Time", "Amount"]]
)

# Train only on genuine transactions
X_normal = X_train[y_train == 0].values
X_test = X_test.values

# 3. Build autoencoder
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_normal.shape[1],)),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(X_normal.shape[1], activation="linear")
])

# 4. Compile model
```

```
model.compile(optimizer="adam", loss="mse", metrics=["mae"])

# 5. Train autoencoder
model.fit(
    X_normal, X_normal,
    epochs=10,
    batch_size=64,
    validation_split=0.1,
    verbose=1
)

# 6. Calculate reconstruction error and threshold
train_pred = model.predict(X_normal, verbose=0)
train_error = np.mean((X_normal - train_pred) ** 2, axis=1)

threshold = np.percentile(train_error, 95)
print("Threshold:", threshold)

# 7. Detect fraud
test_pred = model.predict(X_test, verbose=0)
test_error = np.mean((X_test - test_pred) ** 2, axis=1)

y_pred = (test_error > threshold).astype(int)

# 8. Evaluate model
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("Classification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

3C

```
import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# Step 1: Load Dataset
data = pd.read_csv("F:/DL/LP IV Updated Datasets/AutoEncoder/Energy Consumption/energy
consumption.csv")

# Step 2: Preprocess Data
data = data.ffill().bfill()

X = data.select_dtypes(include=np.number).drop(
    columns=["Anomaly_Label"], errors="ignore"
)

y = data["Anomaly_Label"]

# Step 3: Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Normalize Data
scaler = MinMaxScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Train Using Normal Data Only
X_train = X_train[y_train.values == 0]

# Step 6: Reshape into Sequences of 10 Readings
X_train = X_train[:len(X_train) // 10 * 10]
X_test = X_test[:len(X_test) // 10 * 10]

y_test = y_test.values[:len(X_test)]

X_train = X_train.reshape(-1, 10, X.shape[1])
X_test = X_test.reshape(-1, 10, X.shape[1])
```

```
y_test = y_test.reshape(-1, 10).max(axis=1)

# Step 7: Build LSTM Autoencoder
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(10, X.shape[1])),
    tf.keras.layers.LSTM(16),
    tf.keras.layers.RepeatVector(10),
    tf.keras.layers.LSTM(16, return_sequences=True),
    tf.keras.layers.TimeDistributed(
        tf.keras.layers.Dense(X.shape[1])
    )
])

model.compile(optimizer="adam", loss="mse")

# Step 8: Train Model
model.fit(X_train, X_train, epochs=5, batch_size=32)

# Step 9: Calculate Reconstruction Error
train_pred = model.predict(X_train)
train_error = np.mean((X_train - train_pred) ** 2, axis=(1, 2))

test_pred = model.predict(X_test)
test_error = np.mean((X_test - test_pred) ** 2, axis=(1, 2))

# Step 10: Detect Anomalies
threshold = np.percentile(train_error, 95)

predictions = (test_error > threshold).astype(int)

print("Threshold:", threshold)
print("Predicted Anomalies (0 = Normal, 1 = Anomaly):")
print(predictions)

# # Step 11: Evaluate Model
# print("\nConfusion Matrix:")
# print(confusion_matrix(y_test, y_pred))

# print("\nClassification Report:")
# print(classification_report(y_test, y_pred, zero_division=0))
```

3D

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# Step 1: Load Dataset
data = pd.read_csv("F:/DL/LP IV Updated Datasets/AutoEncoder/NASA Turbofan ENgine/NASA
TurboFan Engine.csv")

# Step 2: Select Sensor Data and Labels
sensor_cols = [f"sensor{i}" for i in range(1, 22)]

X = data[sensor_cols]
y = data["Fault_Label"]

# Step 3: Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 4: Normalize Data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Train Using Healthy Data Only
X_train = X_train[y_train.values == 0]

# Step 6: Reshape Data into Sequences
sequence_length = 10
n_features = 21

# Keep only complete sequences
X_train = X_train[:len(X_train) // sequence_length * sequence_length]
X_test = X_test[:len(X_test) // sequence_length * sequence_length]

y_test = y_test.values[:len(X_test)]
```

```
# Reshape sensor data
X_train = X_train.reshape(-1, sequence_length, n_features)
X_test = X_test.reshape(-1, sequence_length, n_features)

# One label for each test sequence:
# 1 if any reading in the sequence is faulty
y_test = y_test.reshape(-1, sequence_length).max(axis=1)

# Step 7: Build LSTM Autoencoder
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(sequence_length, n_features)),
    tf.keras.layers.LSTM(16),
    tf.keras.layers.RepeatVector(sequence_length),
    tf.keras.layers.LSTM(16, return_sequences=True),
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(n_features))
])

model.compile(optimizer="adam", loss="mse")
# Step 8: Train Model
model.fit(
    X_train, X_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1
)

# Step 9: Calculate Reconstruction Error
train_predictions = model.predict(X_train)
train_errors = np.mean((X_train - train_predictions) ** 2, axis=(1, 2))
test_predictions = model.predict(X_test)
test_errors = np.mean((X_test - test_predictions) ** 2, axis=(1, 2))

# Step 10: Detect Faults Using Threshold
threshold = np.percentile(train_errors, 95)
y_pred = (test_errors > threshold).astype(int)
print("Threshold:", threshold)
print("Predicted Faults (0 = Healthy, 1 = Faulty):")
print(y_pred)

# # Step 11: Evaluate Model
# print("\nConfusion Matrix:")
# print(confusion_matrix(y_test, y_pred))
# print("\nClassification Report:")
# print(classification_report(y_test, y_pred, zero_division=0))
```

3F

```
import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.preprocessing import MinMaxScaler

# Step 1: Load Dataset
folder = "F:/DL/LP IV Updated Datasets/AutoEncoder/Sign Language MNIST/"

train_data = pd.read_csv(folder + "sign_mnist_train.csv")
test_data = pd.read_csv(folder + "sign_mnist_test.csv")

# Step 2: Separate Labels and Images
X_train = train_data.drop("label", axis=1).values
y_train = train_data["label"].values

X_test = test_data.drop("label", axis=1).values
y_test = test_data["label"].values

# Step 3: Normalize Pixel Values
X_train = X_train / 255.0
X_test = X_test / 255.0

# Step 4: Select Label 24 (Y) as Anomaly
X_train_normal = X_train[y_train != 24]

X_test_normal = X_test[y_test != 24]
X_test_anomaly = X_test[y_test == 24]

# Combine normal and anomaly test images
X_test_final = np.concatenate([X_test_normal, X_test_anomaly])
y_test_final = np.concatenate([
    np.zeros(len(X_test_normal)),
    np.ones(len(X_test_anomaly))
])

# Step 5: Build Autoencoder
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(128, activation="relu"),
```

```
    tf.keras.layers.Dense(784, activation="sigmoid")
])

model.compile(optimizer="adam", loss="mse")

# Step 6: Train on Normal Classes Only
model.fit(
    X_train_normal, X_train_normal,
    epochs=5,
    batch_size=32,
    validation_split=0.1
)

# Step 7: Calculate Reconstruction Error
reconstructed = model.predict(X_test_final)

errors = np.mean((X_test_final - reconstructed) ** 2, axis=1)

# Step 8: Set Threshold
train_reconstructed = model.predict(X_train_normal)

train_errors = np.mean(
    (X_train_normal - train_reconstructed) ** 2,
    axis=1
)

threshold = np.percentile(train_errors, 95)

# Step 9: Detect Anomalies
predictions = (errors > threshold).astype(int)

print("Reconstruction Error Threshold:", threshold)
print("\nActual Labels (0 = Normal, 1 = Y Anomaly):")
print(y_test_final.astype(int))

print("\nPredicted Labels (0 = Normal, 1 = Anomaly):")
print(predictions)
```

3_ECG

```
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score
from tensorflow.keras import layers, models

df = pd.read_csv("AutoEncoder/ECG/ecg.csv", header=None)

print("Dataset shape:", df.shape)
print(df.head())

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

x_train = x_train[y_train == 1]

print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)

scaler = MinMaxScaler()

x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

encoder = models.Sequential([
    layers.Input(shape=(x_train.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu")
])

decoder = models.Sequential([
    layers.Input(shape=(8,)),
```

```
    layers.Dense(16, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(x_train.shape[1], activation="sigmoid")
])

model = models.Sequential([encoder, decoder])

model.compile(
    optimizer="adam",
    loss="mean_squared_logarithmic_error",
    metrics=["mse"]
)

history = model.fit(
    x_train, x_train,
    validation_split=0.1,
    epochs=20,
    batch_size=512
)

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSLE Loss")
plt.legend()
plt.show()

predictions = model.predict(x_train, verbose=0)
errors = tf.keras.losses.msle(x_train, predictions).numpy()
threshold = np.mean(errors) + np.std(errors)

predictions = model.predict(x_test, verbose=0)
errors = tf.keras.losses.msle(x_test, predictions).numpy()

anomalies = (errors > threshold).astype(int)
y_pred = 1 - anomalies

print("Threshold:", threshold)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Number of anomalies:", np.sum(anomalies))
```

4A

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from gensim.models import Word2Vec
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

df = pd.read_csv("F:/DL/LP IV Updated Datasets/CBOW/IMDB Dataset.csv")

print(df.head())
print(df.shape)

x = df["review"]
y = df["sentiment"].map({"positive": 1, "negative": 0})

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

train_words = [review.lower().split() for review in x_train]
test_words = [review.lower().split() for review in x_test]

print(train_words[0][:20])

cbow = Word2Vec(
    sentences=train_words,
    vector_size=100,
    window=5,
    min_count=2,
    sg=0,
    epochs=5
)

print("Vocabulary size:", len(cbow.wv))

x_train_embed = np.array([cbow.wv.get_mean_vector(r) for r in train_words])
x_test_embed = np.array([cbow.wv.get_mean_vector(r) for r in test_words])

print(x_train_embed.shape)
```

```
print(x_test_embed.shape)

model = Sequential([
    Dense(64, activation="relu", input_shape=(100,)),
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.summary()

history = model.fit(
    x_train_embed, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)

loss, accuracy = model.evaluate(x_test_embed, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

4B

```
import pandas as pd
import re
from gensim.models import Word2Vec
# Step 1: Load Dataset
df = pd.read_csv("F:/DL/LP IV Updated Datasets/CBOW/twitter_dataset.csv")
print(df.head())
print(df.shape)

# Step 2: Preprocess Tweets
tweets = df["Text"].dropna().astype(str)
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", "", text)
    return text.split()

sentences = [clean_text(tweet) for tweet in tweets]

# Remove empty tweets
sentences = [words for words in sentences if words]
print("Sample tokens:", sentences[0][:10])

# Step 3: Train CBOW Model
model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=1,
    sg=0,
    epochs=5
)
print("Vocabulary Size:", len(model.wv))

# Step 4: Display Word Embedding
word = list(model.wv.index_to_key)[0]

print("\nSelected Word:", word)
print("Word Embedding:")
print(model.wv[word])

# Step 5: Find Similar Words
print("\nWords Similar to", word)
print(model.wv.most_similar(word, topn=5))
```

4C

```
import re
from gensim.models import Word2Vec
# Step 1: Read the Text Document
with open("document3.txt", "r", encoding="utf-8") as file:
    text = file.read()
print("Original Text:\n", text)

# Step 2: Preprocess the Text
text = text.lower()
# Split text into sentences
sentences = re.split(r"[.!?\n]+", text)
# Tokenize and remove special characters
words = [
    re.sub(r"[^a-z\s]", "", sentence).split()
    for sentence in sentences
]
# Remove empty sentences
words = [sentence for sentence in words if sentence]
print("\nTokenized Text:")
print(words)

# Step 3: Train CBOW Model
model = Word2Vec(
    sentences=words,
    vector_size=50,
    window=5,
    min_count=1,
    sg=0,
    epochs=50
)

# Step 4: Display Vocabulary Size
print("\nVocabulary Size:", len(model.wv))

# Step 5: Display Word Embedding
word = "influenza"
print("\nEmbedding of", word, ":")
print(model.wv[word])

# Step 6: Find Similar Words
print("\nWords Similar to", word, ":")
print(model.wv.most_similar(word, topn=5))
```

4D

```
import pandas as pd
import numpy as np
import re

from gensim.models import Word2Vec
from sklearn.metrics.pairwise import cosine_similarity

# Step 1: Load Twitter Dataset
df = pd.read_csv("F:/DL/LP IV Updated Datasets/CBOW/twitter_dataset.csv")

tweets = df["Text"].dropna().astype(str).tolist()

# Step 2: Preprocess Tweets
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", "", text)
    return text.split()

sentences = [clean_text(tweet) for tweet in tweets]

# Remove empty tweets
valid = [(tweet, words) for tweet, words in zip(tweets, sentences) if words]
tweets = [item[0] for item in valid]
sentences = [item[1] for item in valid]

# Step 3: Train CBOW Model
model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=1,
    sg=0,
    epochs=5
)

print("Vocabulary Size:", len(model.wv))

# Step 4: Create Tweet Embeddings
def get_embedding(words):
    vectors = [model.wv[word] for word in words if word in model.wv]

    if len(vectors) == 0:
```

```
        return np.zeros(100)

    return np.mean(vectors, axis=0)

tweet_embeddings = np.array([
    get_embedding(words) for words in sentences
])

# Step 5: Get User Input
query = input("Enter a topic or keyword: ")
query_words = clean_text(query)

query_embedding = get_embedding(query_words)

# Step 6: Calculate Similarity
if not any(word in model.wv for word in query_words):
    print("No matching words found in the vocabulary.")
else:
    similarities = cosine_similarity(
        query_embedding.reshape(1, -1),
        tweet_embeddings
    )[0]

    # Step 7: Recommend Top 3 Tweets
    top_indices = np.argsort(similarities)[::-1][:3]

    print("\nRecommended Tweets:")

    for i in top_indices:
        print("\nTweet:", tweets[i])
        print("Similarity:", round(similarities[i], 3))
```

5A

```
# a. Import packages first
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input


# b. Load Caltech-101 images at VGG16's preferred 224 x 224 size.
size = (224, 224)
train_ds = tf.keras.utils.image_dataset_from_directory(
    "F:/DL/LP IV Updated Datasets/Object Detection/caltech-101", validation_split=0.2,
subset="training", seed=42,
    image_size=size, batch_size=16
)
test_ds = tf.keras.utils.image_dataset_from_directory(
    "F:/DL/LP IV Updated Datasets/Object Detection/caltech-101", validation_split=0.2,
subset="validation", seed=42,
    image_size=size, batch_size=16
)
class_names = train_ds.class_names

# c and d. Load VGG16, freeze it, and add a stronger classifier.
base = VGG16(weights = "F:/DL/LP IV Updated Datasets/Object
Detection/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5", include_top=False,
input_shape=(224, 224, 3))
base.trainable = False
model = models.Sequential([
    layers.Lambda(preprocess_input),
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(len(class_names), activation="softmax")
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

# e and f. Train on more images for better accuracy.
history = model.fit(train_ds.take(100), validation_data=test_ds.take(10), epochs=3)

# g. Evaluate loss and accuracy.
loss, accuracy = model.evaluate(test_ds.take(10))
```

```
print("Loss:", loss)
print("Accuracy:", accuracy)

# h. Show one prediction.
for images, labels in test_ds.take(1):
    prediction = tf.argmax(model.predict(images[:1], verbose=0)[0]).numpy()
    plt.imshow(images[0].numpy().astype("uint8"))
    plt.title("Actual: " + class_names[labels[0]] + " | Predicted: " + class_names[prediction])
    plt.axis("off")
    plt.show()

# i. Fine tune the top VGG16 layers with a small learning rate.
base.trainable = True
for layer in base.layers[:-6]:
    layer.trainable = False
model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(train_ds.take(100), validation_data=test_ds.take(10), epochs=1)

# g. Evaluate loss and accuracy.
loss, accuracy = model.evaluate(test_ds.take(10))
print("Loss:", loss)
print("Accuracy:", accuracy)
```

5B

```
# a. Import packages first
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG16

# b. Load updated Intel class folders and create train/validation datasets.
size = (128, 128)
train_ds = tf.keras.utils.image_dataset_from_directory(
    "Intel Image Classification", validation_split=0.2, subset="training",
    seed=42, image_size=size, batch_size=16
)
test_ds = tf.keras.utils.image_dataset_from_directory(
    "Intel Image Classification", validation_split=0.2, subset="validation",
    seed=42, image_size=size, batch_size=16
)
val_ds = test_ds
class_names = train_ds.class_names

# c and d. Load VGG16, freeze it, and add a small classifier.
base = VGG16(weights="vgg16_notop_weights.h5", include_top=False, input_shape=(128, 128,
3))
base.trainable = False
model = models.Sequential([
    layers.Rescaling(1./255),
    base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(len(class_names), activation="softmax")
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

# e and f. Train and compute loss.
history = model.fit(train_ds.take(50), validation_data=val_ds.take(10), epochs=2)

# g. Evaluate loss and accuracy on test data.
loss, accuracy = model.evaluate(test_ds.take(10))
print("Loss:", loss)
print("Accuracy:", accuracy)

# h. Show one prediction.
```

```
for images, labels in test_ds.take(1):
    prediction = tf.argmax(model.predict(images[:1], verbose=0)[0]).numpy()
    plt.imshow(images[0].numpy().astype("uint8"))
    plt.title("Actual: " + class_names[labels[0]] + " | Predicted: " + class_names[prediction])
    plt.axis("off")
    plt.show()

# i. Fine tune the top VGG16 layers.
base.trainable = True
for layer in base.layers[:-4]:
    layer.trainable = False
model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(train_ds.take(10), validation_data=val_ds.take(10), epochs=1)
```

5C

```
# a. Import packages first
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG16

size = (128,128)

train_ds = tf.keras.utils.image_dataset_from_directory(
    "F:/DL/LP IV Updated Datasets/Object Detection/fashion mnist",
    validation_split = 0.2,
    subset = "training",
    seed = 42,
    image_size = size,
    batch_size = 16,
    color_mode = "rgb"
)

test_ds = tf.keras.utils.image_dataset_from_directory(
    "F:/DL/LP IV Updated Datasets/Object Detection/fashion mnist",
    validation_split = 0.2,
    subset = "validation",
    seed = 42,
    image_size = size,
    batch_size = 16,
    color_mode = "rgb"
)

val_ds = test_ds

class_names = train_ds.class_names

base = VGG16(
    weights = "F:/DL/LP IV Updated Datasets/Object
Detection/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5",
    include_top = False,
    input_shape = (128,128,3)
)

model = models.Sequential([
    layers.Rescaling(1./255),
    base,
```

```
    layers.GlobalAveragePooling2D(),
    layers.Dense(len(class_names), activation= "softmax")
])

model.compile(
    optimizer="adam",
    loss= "sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    train_ds.take(50),
    validation_data= val_ds.take(10),
    epochs = 5
)

loss, accuracy = model.evaluate(test_ds.take(10))
print("Loss: ", loss)
print("Accuracy: ", accuracy)

for image, label in test_ds.take(1):
    prediction = tf.argmax(model.predict(image[:1], verbose=0)[0]).numpy()
    plt.imshow(image[0].numpy().astype("uint8"))
    plt.title("Actual:"+class_names[label[0]]+" | Predicted: "+ class_names[prediction])

    plt.axis("off")
    plt.show()

base.trainable = True
for layer in base.layers[:-4]:
    layer.trainable = False

model.compile(
    optimizer="adam",
    loss= "sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    train_ds.take(50),
    validation_data= val_ds.take(10),
    epochs = 3
)
```

```
final_loss, final_accuracy = model.evaluate(test_ds.take(10))
print("Loss: ", final_loss)
print("Accuracy: ", final_accuracy)
```